Verifier-Assisted versus LLM-Only Pre-deployment Network-Change Validation: A Preregistered Paired Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We measured whether integrating a deterministic pre-deployment verifier with a bounded automated repair budget increases the fraction of intent-driven network-change proposals that pass semantic validation compared with accepting a single LLM candidate. In a preregistered paired design (study id cnsmp2026-llm-verifier-predeployment-001) we executed 300 paired intents (100 per difficulty stratum: low/medium/high) using gpt-5-mini and deterministic_netops_validator_v5. Each paired intent produced one shared_initial generation that was evaluated under two conditions: baseline (accept single shot) and guarded (deterministic validation and <=1 automated repair). Outcomes: baseline 299/300 unflagged passes (0.9966667), guarded 300/300 (1.0); contingency n11=299, n10=1, n01=0, n00=0; absolute paired difference = 0.0033333; exact McNemar two-sided p = 1.0; 95% paired bootstrap percentile CI = [0.00,0.01] (bootstrap_seed = 1729, B = 10000). Under the supplied synthetic suite and repair budget the single-shot generator was near ceiling, limiting observable deterministic rescue. We archive preregistration, execution, and analysis artifacts and interpret results with respect to estimand conditioning, validator coverage, repair budget, and operational deployment policies.**

I. Introduction

Generative large language models (LLMs) are being applied to network operations (NetOps) tasks such as intent→configuration translation, configuration synthesis, and automated repair. For pre-deployment network changes, semantic correctness properties—sequencing constraints, reachability, and policy preservation—are operationally critical and cannot be assured by superficial syntactic checks alone. A common engineering pattern is to pair generative models with deterministic validators and bounded automated repair (generate→validate→repair) to reduce operational risk. However, the measurable benefit of such guarded pipelines depends on the experimental estimand (what is held fixed), the generator single-shot quality, the validator's expressiveness, and the permitted repair budget.

We preregistered a paired experiment (study id cnsmp2026-llm-verifier-predeployment-001) to quantify the deterministic rescue achievable when the same single LLM candidate is evaluated under two treatments. For each intent the pipeline produced exactly one shared_initial candidate using the declared generator (gpt-5-mini) and compared: (a) baseline — accept the single candidate without repair, and (b) guarded — run a deterministic validator (deterministic_netops_validator_v5) and allow at most one automated repair which itself is re-validated. The primary research question: does adding a deterministic verifier plus at most one automated repair increase the probability that a single LLM candidate passes semantic pre-deployment validation compared with accepting that same candidate without repair?

Contributions of this artifact-grounded study: (1) a preregistered paired evaluation isolating deterministic rescue under a shared_initial estimand that conditions on a single LLM draw per intent; (2) a complete execution and analysis bundle including validator traces and the single automated repair transcript archived with the run; and (3) an evidence-grounded interpretation of estimator conditioning, ceiling effects, and operational implications for NetOps pipelines.

II. Related Work

LLMs for NetOps and intent translation have rapidly advanced, producing systems and surveys that document a range of applied tasks (intent→configuration, template synthesis, protocol-test generation, and repair) and highlight practical engineering tradeoffs [https://openalex.org/W4402742293; https://openalex.org/W4411015066; https://openalex.org/W4416927413]. Several recent system papers emphasize tool augmentation: planner-centric and ReAct-style frameworks allow LLMs to call external validators, simulators, or synthesizers to decompose multi-step NetOps workflows and coordinate tool outputs [doi:10.1609/aaai.v40i40.40676; https://openalex.org/W4389518608]. Empirical reports show that introducing deterministic checks and generate→validate→repair scaffolds can materially improve surface correctness metrics relative to unconstrained generation, but the magnitude depends strongly on validator coverage, task distribution, and whether pipelines permit resampling or interactive steering [https://openalex.org/W4404240614; https://openalex.org/W4416927413].

Two methodological themes recur in the literature and motivate our design choices. First, many prior evaluations mix benefits from deterministic validation with benefits from multiple independent model draws or verifier-guided regeneration; this conflation makes it difficult to bound the deterministic rescue attributable to post-generation checking alone [https://openalex.org/W4411015066]. Second, publicly available NetOps benchmarks and task suites often emphasize syntactic or template correctness rather

than semantically rich invariants (reachability, sequencing, policy preservation), limiting operational generality of evaluation claims [https://openalex.org/W4415071524; https://openalex.org/W4392875514]. Work proposing semantic or verifier-centric benchmarks stresses the need for validators that can express sequencing and reachability constraints, but community adoption remains fragmentary [https://openalex.org/W4415071524].

Related system-level work examines hybrid repair and verification: studies combining symbolic fault localization or formal checks with generative repair operators report promising but variable success depending on fault class and verifier expressiveness [https://openalex.org/W4409346645; https://openalex.org/W4404240614]. Complementary research in hallucination detection and deterministic safety nets demonstrates detection approaches (entropy or semantic-consistency detectors, retrieval augmentation, self-reflection) that can reduce false positives but again depend on the threat model and the validator's semantic reach [https://openalex.org/W4399803256; doi:10.2139/ssrn.6538460]. These works collectively underline that validator design (what properties are checkable) and the allowed repair operator set are primary determinants of the guarded pipeline's practical value.

Our study situates itself explicitly within these threads: rather than measuring an open-ended tool-augmented agent that may resample or iteratively regenerate, we isolate deterministic rescue by conditioning on a single shared_initial LLM draw and allowing at most one constrained repair. This isolates the bounded, post-hoc verifier/repair effect from resampling benefits reported elsewhere and provides an operationally meaningful bound for policies that limit model calls. In doing so we follow the literature's call for clearer estimand definition and for evaluations tied to validator semantics and repair budgets [https://openalex.org/W4404240614; doi:10.1609/aaai.v40i40.40676].

III. Methodology

Preregistration and primary estimand. The confirmatory design, analysis plan, inclusion/exclusion rules, and estimand were frozen in a machine-readable preregistration (study id cnsmp2026-llm-verifier-predeployment-001; preregistration SHA-256 = 425cb1652354120cf48f686b964723d64c76d0a8abe53cd5b5ad0eaa1af71fc7). The primary estimand is the absolute paired difference in the proportion of unflagged verifier passes (guarded minus baseline) computed on N = 300 paired intents after applying preregistered exclusion rules. The analysis executor paired_binary_analysis_v1 (frozen) performs contingency counting (n11,n10,n01,n00), an exact two-sided McNemar test, and a paired percentile bootstrap to produce a 95% CI. Bootstrap parameters used in the confirmatory analysis were seed = 1729 and B = 10000 as recorded in the archived analysis artifacts.

Task generation, stratification, and semantics. The task bank was produced deterministically by deterministic_netops_task_generator_v5 into a synthetic_topology_suite stratified into equal-sized low/medium/high difficulty bins. Each task manifest entry encodes: (a) an explicit natural-language intent; (b) initial and expected_final state maps; (c) the required_sequence of field/interface edits that represent sequencing and safety constraints (e.g., must_be_down_for_fields); (d) a compact difficulty fingerprint (changed_interface_count, dependency_rule_count, required_command_count, transient_rule_count); and (e) preserve_unspecified_state and protected_interfaces invariants. The stratification was operationalized by generator metadata so that each stratum contained 100 paired cases in the main run. The synthetic suite implements deterministic reachability and topology invariants that the validator checks; this permits semantic scoring (not just syntactic acceptance).

Shared_initial protocol and generation model control. To isolate deterministic rescue from resampling benefits we implemented a shared_initial estimand: for each paired intent the pipeline produced exactly one shared_initial LLM candidate and reused that identical candidate for both conditions. The declared generator for shared_initial production was gpt-5-mini (execution/model_configuration.json records requested_model = "gpt-5-mini"). The execution contract limited initial_generation_calls_per_task = 1 and maximum_repair_calls_per_task = 1. These settings fix the model-call budget and ensure that any difference between conditions arises only from the deterministic validator/repair stage applied to the same candidate rather than from drawing alternative candidates.

Validator, scoring rule, and constrained repair operator. Semantic validation used deterministic_netops_validator_v5 under the preregistered full_intent_constraint_validation rule. That rule performs deterministic checks for: sequencing constraints (must_be_down_for_fields), required_command_count equality, preservation of unspecified state, forbidden-template protections (protected_interfaces), and a pre/post reachability invariant test encoded by the synthetic topology suite. Scoring was binary: an unflagged result (valid == true) denotes a pass; any violation or missing required command yields an automated_flag and counts as a failure for that pipeline call.

The guarded condition permitted at most one automated repair (repair budget <=1). The repair operator implemented a constrained edit policy restricted to localized, intent-conservative fixes: insertion of missing sequencing commands, reordering of adjacent commands to satisfy explicit required_sequence constraints, or minor syntactic normalizations that do not alter fields outside the allowed_touched_fields. Repairs were deterministically proposed and applied, then re-validated by the same validator; if the repaired candidate remained flagged it counted as a failure. The repair provider trace (archived) documents the exact edit and subsequent validator outcome for each repair invocation (the run used one repair call). The repair operator intentionally excluded large-scope rewrites or generator re-invocations to preserve the shared_initial estimand.

Contamination controls, negative controls, and stopping rules. The preregistration mandated one negative-control (in-

tentionally malformed) task per difficulty stratum to exercise the contamination detectors and invariant checks. Contamination detection included deterministic invariant comparisons (pre/post reachability matrices) and negative-control pass/fail checks; items that passed a negative control would be flagged as contamination (none did). Exclusion rules: a pair is excluded from confirmatory analysis only if both pipeline calls for that pair fail due to provider/infrastructure outage; single-call failures are handled per the executor's complete_pair_primary rule (none occurred). The stopping rule would halt the run if >20% of attempted pairs were excluded due to dual-call outages; this did not trigger in execution.

Execution instrumentation, audit, and deterministic reconciliation. The run used adapter hosted_netops_gvr_v1 in execution_mode = scientific_confirmatory. Execution manifests record planned_episode_count = 600, completed_episode_count = 600, complete_pair_count = 300, model_calls_used = 301, and maximum_model_calls = 600. Provider auditing values archived with the run include hosted_model_call_event_count = 301, cache_miss_count = 301, and stage_counts showing shared_initial_generation = 300 and repair = 1. Deterministic reconciliation procedures verified pair accounting and call auditing and report all_deterministic_consistency_checks_passed = true. The execution and reconciliation artifacts (execution/raw_results.jsonl, execution/task_manifest.jsonl, execution/model_configuration.json, and execution/execution_log.jsonl) are archived to permit third-party verification of accounting and reproducibility.

Analysis executor and inferential controls. The primary confirmatory analysis followed the preregistered paired_binary_analysis_v1 executor: compute contingency counts (n11,n10,n01,n00), absolute paired difference (guarded - baseline), perform an exact two-sided McNemar test for paired binary outcomes, and construct a paired percentile bootstrap 95% CI (seed = 1729, B = 10000). Secondary preregistered sensitivity analyses (failure-as-zero, stratum-wise paired tests with Benjamini–Hochberg FDR correction) were run as exploratory diagnostics; confirmatory claims rely only on the primary analysis. The analysis pipeline recorded and archived its outputs (analysis/paired_contingency_table.csv, analysis/condition_summary.csv, analysis/missingness_summary.csv) and a reproducible analysis log (analysis/analysis_log.jsonl).

Missingness, exclusions, and reproducibility artifacts. The preregistered missingness plan required logging of provider/API technical failures and explicit treatment rules; no pairs were excluded under those rules in the confirmatory run. All raw responses, validator traces, repair traces, provider call traces, task manifests, and analysis artifacts referenced above are archived with machine-readable hashes. The archived initial_master_prompt reference path and SHA-256 (provenance/master_prompt.txt, SHA-256 = 1872df1e1805d2d9 6940456ca016bd665d1d5196add77f5acdf1582bb39b15ba) are included in the run bundle and the Disclosure Statement. The preregistration described an optional pilot (n = 60) for discordance estimation; that pilot was not executed prior to the main run (this deviation is recorded in the preregistration and Disclosure).

IV. Results

Execution integrity and accounting. The preregistered confirmatory run completed as planned with completed_episode_count = 600 (300 paired intents reused as shared_initial candidates) and complete_pair_count = 300. Provider auditing recorded hosted_model_call_event_count = 301 (300 shared_initial generations plus one repair invocation) and model_calls_used = 301; stage_counts indicate shared_initial_generation = 300 and repair = 1. Deterministic reconciliation confirms that all deterministic consistency checks passed (all_deterministic_consistency_checks_passed = true) and that no provider/infrastructure outages caused exclusions under the preregistered rules.

Primary confirmatory outcomes. Aggregate condition totals (analysis/condition_summary.csv) show baseline successes = 299/300 (0.9966666666666667) and guarded successes = 300/300 (1.0). The paired contingency table (analysis/paired_contingency_table.csv) is n11 = 299 (both pass), n10 = 1 (guarded pass only), n01 = 0 (baseline pass only), n00 = 0 (both fail). The absolute paired difference (guarded - baseline) is 0.0033333333333332993. The preregistered exact McNemar two-sided test returned p = 1.0, reflecting the extremely small number of discordant pairs. The paired bootstrap percentile 95% confidence interval (bootstrap_seed = 1729, B = 10000) reported with the analysis executor is [0.00, 0.01], bounding the plausible absolute paired improvement under the sampled suite and design.

Per-stratum diagnostics. Stratum-wise archived totals (strata defined in the task manifest by difficulty metadata) reveal: low difficulty — baseline 100/100, guarded 100/100; medium — baseline 100/100, guarded 100/100; high — baseline 99/100, guarded 100/100. The single discordant pair originates from the high-difficulty stratum. These per-stratum counts are provided as exploratory diagnostics in the archived artifacts and confirm that the limited discordance was concentrated in the highest difficulty bin in this synthetic suite.

Repair diagnostic and episode-level trace. The lone discordant episode (the single n10) triggered the prerogative repair stage. Validator traces for that episode attribute the initial flag to an omitted sequencing command required by the intent (a must_be_down_for_fields sequencing violation): the shared_initial candidate lacked the mandated 'admin down' step prior to a VLAN/MTU change. The constrained automated repair applied a single localized insertion of the missing sequencing command; deterministic_netops_validator_v5 subsequently reported valid == true for the repaired candidate. The repair provider trace and the validator trace for this episode are archived with the run; the manuscript reports this concise semantic summary rather than reproducing raw provider transcripts in the body.

Contamination and missingness. The contamination_summary.csv is empty (no contamination flags recorded), and the missingness summary (analysis/missingness_summary.csv) reports zero failed or unscorable episodes and zero excluded pairs under the preregistered exclusion rules. No pairs were removed for dual-call provider outages, and no single-call failures occurred that required sensitivity-analysis handling.

Statistical context and practical accounting. The combination of high baseline single-shot success (299/300) and a bounded repair budget (<=1 edit per flagged candidate) produced only a single deterministic rescue. Provider-call accounting shows that achieving that rescue required one additional model-stage call beyond the 300 shared_initial generations (total model_calls_used = 301), indicating the marginal model-call cost of the constrained repair policy in this run. All analysis artifacts (contingency table, condition summary, bootstrap parameters, and reconciliation reports) are archived for reproducibility and forensic inspection.

V. Discussion

Estimand conditioning and operational alignment. Conditioning on a single LLM draw per intent (shared_initial) corresponds to operational policies that constrain model calls for cost, latency, or governance reasons. Under such policies the guarded pipeline measures only deterministic validator and bounded repair rescue applied to the same candidate. Workflows that permit resampling, verifier-guided generation, or interactive steering change the estimand and typically increase achievable success rates by exploiting generator variance; those workflows must be evaluated under a different preregistered estimand.

Ceiling effects and the omitted pilot. The preregistration targeted detection of a 10 percentage-point absolute improvement with planned power $\approx$0.8 based on a pilot to estimate discordance. Because the optional pilot (n = 60) intended to estimate discordance was not executed, the main run confronted a near-ceiling baseline (299/300), producing minimal discordance (m = 1) and reduced power to detect small absolute differences. This practical outcome highlights the utility of small calibration pilots when generator performance may approach ceiling on chosen task suites.

Validator coverage and failure-mode dependence. The observed single failure mode was a sequencing omission; other classes (reachability violations, forbidden ACL removal) were not observed at scale on this synthetic suite. The marginal utility of a guarded pipeline therefore depends on (a) the deployment distribution of failure modes and (b) validator expressiveness: richer validators that detect reachability or ACL semantics may increase observed rescue rates. Conversely, if operator risk policies disallow automated edits, the guarded pipeline's deterministic rescue becomes operationally limited to detection and human escalation.

Repair budget and bounded edits. We intentionally limited automated repair to a single constrained edit to measure bounded deterministic rescue. That design measured only localized sequencing corrections; iterative repair, larger edit operators, or interactive verifier-guided generation would increase rescue opportunities but address a different estimand. The archived single repair trace enables forensic analysis of edit efficacy and could inform permitted edit-operator design in production pipelines.

Practical implications for NetOps CI/CD. Organizations must align evaluation estimands with operational choices: if pipelines permit multiple model calls or verifier-guided regeneration, evaluations should allow resampling and interactive steering to measure the achievable success rates under those policies. If model calls are tightly budgeted, the shared_initial estimand provides an operationally meaningful upper bound for deterministic verifier/repair rescue applied to a single candidate. Our artifact audit (model_calls_used = 301, repair_stage_count = 1) shows that in this run bounded repair produced measurable rescue at very low additional model cost for the single rescued episode.

VI. Limitations

- Synthetic benchmark: tasks and topologies were generated by the preregistered deterministic task generator; real-world heterogeneity may expose different failure modes and increase verifier marginal utility.
- Single model and validator: only gpt-5-mini and deterministic_netops_validator_v5 were evaluated; results may not generalize to other LLMs or richer/diverse validators.
- Ceiling effect: baseline single-shot performance was near 100% on this suite (299/300), producing limited discordance and reduced power to detect small absolute improvements relative to larger pre-specified targets.
- Pilot not executed: the preregistered pilot intended for discordance estimation and power calibration was not run; no post-preregistration power recalculation was performed.
- Estimand conditioning: the shared_initial protocol conditions the estimand on a single LLM candidate and therefore does not measure verifier benefit that would arise from allowing independent alternative LLM candidates per condition (resampling or iterative generation).
- Repair budget: the guarded condition permitted at most one automated repair per pair; larger or iterative repair strategies may yield different outcomes.

VII. Conclusion

We executed a preregistered, artifact-archived paired comparison between a verifier-assisted pipeline (deterministic validator + <=1 automated repair) and accepting a single-shot LLM candidate under the shared_initial estimand on 300 synthetic NetOps intents using gpt-5-mini and deterministic_netops_validator_v5. Baseline single-shot generation produced 299/300 unflagged verifier passes and the guarded pipeline 300/300, yielding an absolute paired improvement of 0.00333 (exact McNemar two-sided p = 1.0; 95% bootstrap CI [0.00,0.01]). Deterministic validator+repair rescued a single sequencing omission under the bounded repair budget. The

near-ceiling baseline limits inferential sensitivity to small absolute improvements under the shared_initial estimand. We release full preregistration, execution, and analysis artifacts to support reproducibility and targeted follow-up studies that vary estimand conditioning, model strength, task distribution, or repair budget.

Disclosure Statement

This experiment and manuscript were produced by an autonomous CNSM Agentic-AI research run executed under the archived initial master prompt (provenance/master_prompt.txt, SHA-256 = 1872df1e 1805d2d96940456ca016bd665d1d5196add77f5acdf1582b b39b15ba). Human role: the human Research Director selected the topic family (Generative AI and LLMs for NetOps) and approved pre-lock infrastructure; all literature search after the autonomy lock, autonomous study selection, preregistration (study id cnsmp2026-llm-verifier-predeployment-001), execution, analysis, manuscript drafting, AI peer review, and final compilation were performed autonomously under the locked master prompt. Models and orchestration: the declared generation model used was gpt-5-mini (execution/model_configuration.json declares requested_model = "gpt-5-mini"); adapter/orchestration framework: hosted_netops_gvr_v1. Validator and tooling: deterministic_netops_validator_v5 performed semantic and syntactic validation according to the preregistered full_intent_constraint_validation rule; the guarded condition permitted at most one automated repair call per pair implemented as a constrained edit operator. Preregistration: the confirmatory design, estimand, analysis executor, exclusion/missingness rules, and multiplicity plan were frozen prior to the main run and archived with the study id (preregistration SHA-256 = 425cb1652354120c f48f686b964723d64c76d0a8abe53cd5b5ad0eaa1af71fc7). Execution summary (archived): planned_episode_count = 600, completed_episode_count = 600, complete_pair_count = 300, model_calls_used = 301; provider audit: hosted_model_call_event_count = 301, stage_counts show shared_initial_generation = 300 and repair = 1. Pilot: the preregistration described an optional pilot (n = 60) for discordance estimation and power calibration; that pilot was not executed and no post-preregistration recalculation of required sample size was performed. Deviations: no post-preregistration changes to the scientific design were made; no pairs were excluded. Human editing: no human manually edited or rewrote technical manuscript content after the autonomy lock. Artifact availability: all raw outputs, logs, task manifests, validator traces, repair provider traces, and analysis artifacts referenced in this manuscript are archived in the run bundle associated with the study id. The archived run bundle contains the low-level repair provider trace documenting the single automated repair and subsequent validation pass; this manuscript reports a concise semantic summary of that repair in Results rather than reproducing full verbatim provider transcripts in the body.

References

[1] Yajie Zhou, Kevin Hsieh, Sathiya Kumaran Mani, Srikanth Kandula, Zaoxing Liu, "MeshAgent: Enabling Reliable Network Management with Large Language Models", 2025, 10.1145/3771567

[2] Xiaolong Wei, Yuehu Dong, Xingliang Wang, Xingyu Zhang, Zhejun Zhao, Dongdong Shen, Long Xia, Dawei Yin, "Beyond ReAct: A Planner-Centric Framework for Complex Tool-Augmented LLM Reasoning", 2026, 10.1609/aaai.v40i40.40676

[3] Xu Liu, Peng Zhang, Anubhavnidhi Abhashkumar, Jiawei Chen, Weirong Jiang, "Automatic Configuration Repair", 2024, 10.1145/3696348.3696895

[4] Hao Zhou, Chengming Hu, Ye Yuan, Yufei Cui, Yili Jin, Can Chen, Haolun Wu, Dun Yuan, Li Jiang, Di Wu, Xue Liu, Jianzhong Zhang, Xianbin Wang, Jiangchuan Liu, "Large Language Model (LLM) for Telecommunications: A Comprehensive Survey on Principles, Key Techniques, and Opportunities", 2024, 10.1109/comst.2024.3465447

[5] X X Zhang, Xianming Gao, Peilin Tao, Tao Feng, "GraphSynth: Synthesis of Network Configuration Templates Using Large Language Models", 2025, 10.1145/3728725.3728742

[6] Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, Yarin Gal, "Detecting hallucinations in large language models using semantic entropy", 2024, 10.1038/s41586-024-07421-0

[7] Yunze Wei, Wei, Kaiwen, Shaohui Du, Wang, Jianyu, Liu, Zhangzhong, Yawen Wang, Zhenxin Li, Congcong Miao, Xiaohui Xie, Yong Cui, "Automated Network Protocol Testing with LLM Agents", 2025, 10.48550/arxiv.2510.13248

[8] Fuliang Li, Haozhi Lang, Jiajie Zhang, Jiaxing Shen, Xingwei Wang, "PreConfig: A Pretrained Model for Automating Network Configuration", 2024, 10.48550/arxiv.2403.09369